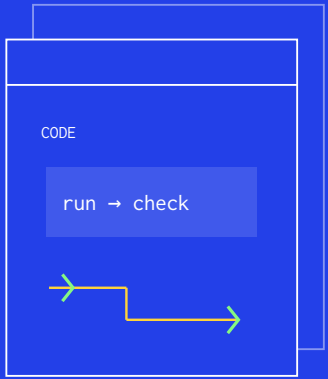
WEEK 2 / LAB 4 / TEAM WORK

MLOps: Machine Learning Operations

# Serve the model. Fail honestly.

Expose the evaluated bike-demand model through a validated API and verify it in the container. Needs Lab 3 merged on main (or develop).



# Done means checked behaviour, not a running process

Three outcomes you can demonstrate from the team repository.

1 **Serve**

Load the saved, evaluated model at startup and return its actual version. Never train inside the service.

2 **Validate**

Reject missing fields, wrong types and out-of-range values before inference.

3 **Verify**

Automated tests cover normal, invalid and missing-model cases, and they run inside the container.

# Who does what

The split is yours; every member owns something and reviews something.

INDIVIDUAL

## One PR each

One area each: validation, load + health/predict, tests, or container + README.

TEAM

## One service

Agree contract, merge order and the final container check.

REVIEW

## Another member

Break a PR you did not write: one bad request, one missing model.

Individual report reports/week-2.md covers Labs 3–4 in your personal repository.

# Add these to the team repository

Paths are the contract; contents are your design.

REQUIRED

Reviewed PRs into your integration branch. Declare new dependencies in pyproject.toml and keep uv.lock in step.

TEAM REPOSITORY PATHS

```
src/bike_demand/api.py
tests/test_api.py
Dockerfile
pyproject.toml  uv.lock
README.md
team report (your format)
```

README: how to run and test the service. Report: add a Lab 4 section.

# Two endpoints, three outcomes

Both share one model-availability state.

GET /health

## 200 or 503

200 only when a usable model is loaded; otherwise 503.

POST /predict

## 200 + version

JSON with a finite non-negative prediction and the loaded model_version.

OTHERWISE

## 4xx or 503

Invalid input gets a 4xx; valid input without a model gets 503.

model_version comes from the artifact's own metadata, not a hard-coded string.

# Input contract: twelve fields

One flat JSON object with the same fields used in training.

RANGES

Weather values are normalised, not Celsius or percentages. cnt, casual and registered are never accepted as inputs.

REQUEST FIELDS

```
season 1-4      yr 0-1
mnth 1-12       hr 0-23
holiday 0-1     weekday 0-6
workingday 0-1 weathersit 1-4
temp atemp hum windspeed
  finite numbers in 0..1
```

Decide and document how extra fields are handled.

# Example: call a service and read the answer

A toy /score endpoint; the shape of the exchange is what matters.

ILLUSTRATIVE EXAMPLE

A valid request returns 200 and JSON. A bad value is rejected with 422 before any model runs.

HTTP exchange

```
curl -X POST localhost:8000/score \
  -H "Content-Type: application/json" \
  -d '{"size": 4}'

200  {"score": 0.82, "model_version": "m-01"}
422  {"detail": [{"loc": ["body", "size"]}]}
```

Different project on purpose: read it, then adapt it to yours.

# Tools you will meet

Lecture 4 covered API concepts; these are this lab's concrete tools.

BUILD

## FastAPI + Pydantic

Routes, request models with field constraints, and an OpenAPI page at /docs.

TEST

## pytest + TestClient

Call the app without a server; start each case in the state it tests.

RUN

## uvicorn + docker

Listen on 0.0.0.0 and publish a port to reach it.

Read the official documentation for each; choosing how to use them is the lab.

# Example: a request model and an endpoint

FastAPI reads the type hints; Pydantic enforces the constraints.

ILLUSTRATIVE EXAMPLE

A value outside the range is rejected before your code runs. Open /docs to see the generated contract.

app.py

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
class Item(BaseModel): size: int = Field(ge=0, le=10)
app = FastAPI()
```

Shown as one-liners to fit the slide; in a real file the class body is indented on its own line.

# Example: answer 503 when the model is missing

Fail with an honest status instead of guessing a number.

ILLUSTRATIVE EXAMPLE

Raise an HTTPException with 503. How you hold and check the model state is your design choice.

app.py

```
from fastapi import HTTPException
state = {"model": None}
def stop(): raise HTTPException(503, "no model")
@app.post("/score")
def score(i: Item): return state["model"] or stop()
```

A health route reads the same state and returns 200 or 503.

# Example: test the service without a server

TestClient calls the app in-process; no port is needed.

ILLUSTRATIVE EXAMPLE

One case per behaviour. Put these lines inside a function named test_something; pytest collects it.

test_app.py

```python
from fastapi.testclient import TestClient
from app import app
client = TestClient(app)
bad = {"size": 40}
r = client.post("/score", json=bad)
assert r.status_code == 422
```

TestClient needs httpx; add it with uv add --dev httpx so the lock stays in step.

# Cheat sheet: HTTP status codes you will use

The status class tells the client who needs to act.

2XX / 4XX / 5XX

4xx: the request is wrong. 5xx: the service cannot answer. 422 is the FastAPI default for invalid input.

status codes

```
200 ok
201 created
400 bad request
404 not found
422 invalid input
500 server error
503 not ready
```

503 is temporary: the service is up but cannot serve yet, for example with no model loaded.

# Cheat sheet: pytest from the terminal

Run all tests, select some, stop early, rerun failures.

RUN AND SELECT TESTS

Use a with-block around TestClient so startup and shutdown (lifespan) run. Start each test in the state it targets.

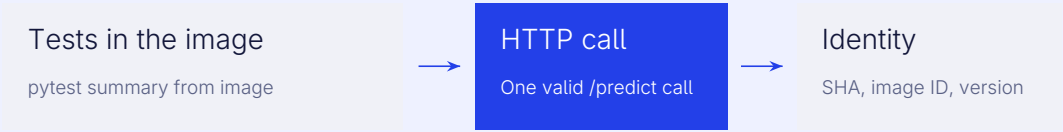
pytest

```
$ uv add --dev httpx pytest
$ uv run pytest -q
$ uv run pytest -k name
$ uv run pytest -x --lf
```

-k filters by name, -x stops at the first failure, --lf reruns the last failures.

# Prove it where it will run

Evidence comes from the built image, not from your laptop environment.




`A passing local pytest does not show the container tests ran.`

Keep raw logs your report can link to; at most three screenshots.

# Example: run the service from its image

Same image, two different commands.

ILLUSTRATIVE EXAMPLE

Listen on 0.0.0.0 inside the container and publish the port to your own machine only.




The start command (for example uvicorn app:app --host 0.0.0.0) lives in your Dockerfile CMD.

# Cheat sheet: git, before and after the tag

Confirm the revision you are about to point at.

CHECK, TAG, PUSH

A tag points at one commit: check that it is reviewed and the tree is clean. Use main instead of develop if that is your branch.

git

```
$ git switch develop
$ git status --short
$ git log --oneline -5
$ git tag -a m2 -m "Week 2"
$ git push origin m2
$ git show m2 --stat
```

Annotated tags (-a) carry a message and a date; submit the tag link with a short note.

# m2 and your individual report

One team checkpoint for Labs 3–4; one personal report.

TEAM

## Annotated tag m2

On your integration branch after review and checks; submit the link and a note.

PERSONAL

## reports/week-2.md

Your contribution, one decision, one blocker, assistance used.

RULES

## Honest and on time

Personal main, no PR or CI. Sunday 23:59. 10% of course.

Incomplete work is reported as incomplete; never invent a result or a passing check.

EXIT CHECK

# A usable service tells the truth when it fails.

Next, Lab 5: local deployment, structured logs and serving-version checks.

# Where the examples come from

Condensed from official documentation; read the originals for every option.

REFERENCES

Commands are shortened and regrouped for this lab. Check the docs when a flag is unclear.




Illustrative examples; run them in your own project before you rely on them.